

# JavaScript Numbers & Numeric Conversion

JavaScript-এ numbers নিয়ে কাজ করার সময় শুধু `Number()` বা `parseInt()` জানা যথেষ্ট না। User input, API data, form values, calculations, decimal precision, `NaN`, `Infinity`—সবকিছু বুঝতে হলে JavaScript-এর Number system এবং conversion rules পরিষ্কারভাবে জানা দরকার।

## 1. JavaScript `number` Type

JavaScript-এ সাধারণ integer এবং floating-point number—দুটোই একই `number` type-এর মধ্যে পড়ে।

```
let age = 25;
let price = 99.99;
let temperature = -10;

console.log(typeof age);           // "number"
console.log(typeof price);         // "number"
console.log(typeof temperature);   // "number"
```

অর্থাৎ:

```
10
10.5
-20
0
0.99
```

সবগুলোই `number`।

## 2. Integer vs Floating-Point Number

### Integer

Decimal part নেই।

```
let age = 30;
let quantity = 5;
let score = 100;
```

# Floating-Point

Decimal part আছে।

```
let price = 99.99;
let rating = 4.5;
let temperature = 36.6;
```

JavaScript-এর জন্য দুটোই:

```
typeof 10;    // "number"
typeof 10.5;  // "number"
```

## 3. Number তৈরি করার বিভিন্ন উপায়

সাধারণভাবে:

```
let a = 100;
let b = 10.5;
let c = -20;
```

এছাড়াও numeric string থেকে number তৈরি করা যায়।

```
let value = "100";

let result = Number(value);

console.log(result);          // 100
console.log(typeof result);  // "number"
```

## 4. `Number()` — Explicit Conversion

`Number()` ব্যবহার করে কোনো value-কে number-এ convert করা যায়।

```
Number("100"); // 100
Number("10.5"); // 10.5
Number(true); // 1
Number(false); // 0
Number(null); // 0
```

কিন্তু invalid numeric string হলে:

```
Number("hello"); // NaN
```

## 5. `Number()` এবং Empty String

এখানে একটি important behavior আছে:

```
Number(""); // 0
```

Space-only string-ও:

```
Number(" "); // 0
```

কারণ whitespace remove করার পর string empty হয়ে যায় এবং `Number()` সেটাকে `0` হিসেবে নেয়।

## 6. `Number()` — Decimal Numbers

```
let price = Number("99.99");

console.log(price); // 99.99
```

এখানে পুরো decimal value convert হয়।

## 7. `Number()` — Invalid Input

```
let value = Number("123abc");

console.log(value); // NaN
```

কারণ পুরো string-টি valid number নয়।

আর:

```
Number("100abc"); // NaN
```

## 8. `parseInt()`

`parseInt()` string থেকে integer বের করার জন্য ব্যবহৃত হয়।

```
parseInt("100"); // 100
parseInt("25"); // 25
```

Decimal থাকলে decimal অংশ বাদ দেয়:

```
parseInt("10.99"); // 10
```

## 9. `parseInt()` — String-এর শুরু থেকে Number পড়া

`parseInt()` string-এর শুরু থেকে যতক্ষণ valid integer পায়, ততক্ষণ পড়ে।

```
parseInt("100px"); // 100
```

এটা:

```
Number("100px"); // NaN
```

থেকে আলাদা।

## Difference

```
Number("100px");  
// NaN  
  
parseInt("100px");  
// 100
```

এটা web development-এ খুব useful হতে পারে।

## 10. `parseInt()` কোথায় থেমে যায়?

```
parseInt("123abc456");  
// 123
```

কারণ শুরুতে `123` valid integer।

`abc` আসার পর parsing বন্ধ হয়ে যায়।

আর:

```
parseInt("abc123");  
// NaN
```

কারণ শুরুতেই valid number নেই।

## 11. `parseInt()` এবং Radix

`parseInt()` -এর দ্বিতীয় argument হলো `radix`।

```
parseInt("101", 2);
```

এখানে `2` মানে binary।

Result:

```
5
```

কারণ:

```
101₂ = 51₀
```

Decimal numbers-এর জন্য সাধারণত:

```
parseInt("100", 10);
```

লেখা ভালো practice।

## 12. `parseFloat()`

`parseFloat()` floating-point number parse করার জন্য ব্যবহৃত হয়।

```
parseFloat("10.50");  
// 10.5
```

আর:

```
parseFloat("99.99");  
// 99.99
```

## 13. `parseFloat()` এবং Extra Characters

`parseFloat()` -ও শুরু থেকে valid numeric অংশ পড়ে।

```
parseFloat("99.99px");  
// 99.99
```

কিন্তু:

```
parseFloat("px99.99");  
// NaN
```

## 14. `Number()` vs `parseInt()` vs `parseFloat()`

| Input     | <code>Number()</code> | <code>parseInt()</code> | <code>parseFloat()</code> |
|-----------|-----------------------|-------------------------|---------------------------|
| "100"     | 100                   | 100                     | 100                       |
| "10.50"   | 10.5                  | 10                      | 10.5                      |
| "100px"   | NaN                   | 100                     | 100                       |
| "99.99px" | NaN                   | 99                      | 99.99                     |
| "hello"   | NaN                   | NaN                     | NaN                       |
| ""        | 0                     | NaN                     | NaN                       |

সহজভাবে মনে রাখো

`Number()`

→ পুরো value-কে number বানানোর চেষ্টা করে

`parseInt()`

→ integer অংশ parse করে

`parseFloat()`

→ decimal number parse করে

## 15. User Input কেন String হয়?

Browser-এর অনেক input value defaultভাবে string দেয়।

ধরো:

```
<input id="age" type="number" />
```

JavaScript:

```
const age = document.querySelector("#age").value;

console.log(typeof age);
```

Result সাধারণত:

```
string
```

যদিও input-এ 25 লেখা হয়েছে।

তাই calculation করার আগে conversion দরকার হতে পারে:

```
const age = Number(  
  document.querySelector("#age").value  
);
```

## 16. Real-World Example — Price Calculation

ধরো API বা form থেকে price এসেছে:

```
const price = "500";  
const quantity = "3";
```

এভাবে করলে:

```
console.log(price + quantity);
```

Result:

```
5003
```

কারণ দুটোই string।

সঠিক:

```
const total = Number(price) * Number(quantity);  
  
console.log(total);  
// 1500
```

## 17. Unary + দিয়ে Number Conversion

`Number()` -এর ছোট alternative হিসেবে unary `+` ব্যবহার করা যায়।

```
const value = "100";

const number = +value;

console.log(number);           // 100
console.log(typeof number);    // "number"
```

আর:

```
+"10.5";
// 10.5

+"hello";
// NaN
```

তবে beginner code-এ readability-এর জন্য:

```
Number(value)
```

অনেক সময় বেশি পরিষ্কার।

---

## 18. `toFixed()`

`toFixed()` decimal places control করার জন্য ব্যবহৃত হয়।

```
const price = 99.4567;

console.log(price.toFixed(2));
// "99.46"
```

এখানে `2` মানে decimal-এর পরে 2টি digit।

---

## 19. `toFixed()` String Return করে

এটা খুব important।

```
const price = 99.4567;

const result = price.toFixed(2);

console.log(result);
console.log(typeof result);
```

Result:

```
99.46
string
```

অর্থাৎ:

```
typeof price.toFixed(2);
// "string"
```

---

## 20. `toFixed()` — বিভিন্ন Examples

---

```
const number = 12.34567;

console.log(number.toFixed(0));
// "12"

console.log(number.toFixed(1));
// "12.3"

console.log(number.toFixed(2));
// "12.35"

console.log(number.toFixed(3));
// "12.346"
```

এটি rounding করে।

---

## 21. `toFixed()` কেন ব্যবহার করা হয়?

বিশেষ করে money/currency display-এর ক্ষেত্রে:

```
const price = 1500;

console.log(price.toFixed(2));
// "1500.00"
```

UI-তে:

₳1500.00

দেখাতে চাইলে useful।

## 22. `toFixed()` এবং Calculation

একটা common mistake:

```
const price = 99.99;

const result = price.toFixed(2);

console.log(result + 10);
```

এখানে unexpected result হতে পারে কারণ `result` string।

Better:

```
const result = Number(price.toFixed(2));

console.log(result + 10);
```

তবে সাধারণত calculation শেষে formatting করা ভালো:

```
const total = price + 10;

console.log(total.toFixed(2));
```

## 23. Number Precision

---

JavaScript `number` internally IEEE 754 double-precision floating-point format ব্যবহার করে।

এর কারণে কিছু decimal calculation দেখতে অদ্ভুত হতে পারে।

```
console.log(0.1 + 0.2);
```

Result:

```
0.30000000000000004
```

এটা JavaScript-এর bug নয়।

এটা binary floating-point representation-এর limitation।

---

## 24. Floating-Point Precision Problem

---

```
console.log(0.1 + 0.2 === 0.3);  
// false
```

তাই decimal calculation-এর ক্ষেত্রে সাবধান হতে হয়।

---

## 25. Practical Floating-Point Solution

---

Display-এর ক্ষেত্রে:

```
const result = 0.1 + 0.2;  
  
console.log(result.toFixed(2));  
// "0.30"
```

Comparison-এর ক্ষেত্রে tolerance ব্যবহার করা যায়:

```
const a = 0.1 + 0.2;

const b = 0.3;

const isEqual = Math.abs(a - b) < Number.EPSILON;

console.log(isEqual);
```

## 26. `Number.EPSILON`

`Number.EPSILON` খুব ছোট একটি value, যা floating-point comparison-এ useful।

```
console.log(Number.EPSILON);
```

তবে মনে রাখতে হবে, সব ধরনের decimal comparison-এর জন্য শুধু `Number.EPSILON` ব্যবহার করলেই যথেষ্ট নয়। বড় সংখ্যার ক্ষেত্রে appropriate tolerance দরকার হতে পারে।

## 27. `NaN`

`NaN` মানে:

*Not a Number*

যদিও interesting ব্যাপার হলো:

```
typeof NaN;
// "number"
```

Example:

```
Number("hello");
// NaN
```

## 28. NaN হলো Invalid Numeric Result-এর Signal

```
const result = Number("abc");

console.log(result);

// NaN
```

এখানে conversion failed করেছে।

## 29. NaN নিজে থেকে Number নয়?

NaN technically number type-এর একটি special value।

```
typeof NaN;

// "number"
```

তাই typeof দিয়ে NaN detect করা যায় না।

ভালো:

```
Number.isNaN(value);
```

## 30. Number.isNaN()

```
Number.isNaN(NaN);

// true

Number.isNaN(100);

// false

Number.isNaN("hello");

// false
```

এটি strictভাবে check করে value আসলেই NaN কিনা।

## 31. Global `isNaN()` বনাম `Number.isNaN()`

Global:

```
isNaN("hello");  
// true
```

কারণ global `isNaN()` আগে value-কে number-এ convert করার চেষ্টা করে।

কিন্তু:

```
Number.isNaN("hello");  
// false
```

কারণ `"hello"` নিজে `NaN` নয়; এটা একটি string।

মনে রাখো

```
Number.isNaN()  
→ actual NaN কিনা check করে  
  
isNaN()  
→ আগে number conversion করার চেষ্টা করে, তারপর check করে
```

## 32. `Infinity`

JavaScript-এ positive infinity:

```
Infinity
```

Example:

```
console.log(1 / 0);  
// Infinity
```

## 33. -Infinity

```
console.log(-1 / 0);  
// -Infinity
```

এগুলোও `number` type-এর।

```
typeof Infinity;  
// "number"
```

## 34. `Number.isFinite()`

কোনো value finite number কিনা check করতে:

```
Number.isFinite(100);  
// true  
  
Number.isFinite(Infinity);  
// false  
  
Number.isFinite(-Infinity);  
// false  
  
Number.isFinite(NaN);  
// false  
  
Number.isFinite("100");  
// false
```

এখানে `"100"` string হওয়ায় false।

## 35. `Number.isInteger()`

কোনো value integer কিনা:

```
Number.isInteger(10);  
    // true  
  
Number.isInteger(10.5);  
    // false  
  
Number.isInteger(-5);  
    // true
```

String হলে:

```
Number.isInteger("10");  
    // false
```

---

## 36. Safe Integer

---

JavaScript-এর `number` সব integer-কে unlimited precision-এ represent করতে পারে না।

Safe integer range:

```
Number.MIN_SAFE_INTEGER  
Number.MAX_SAFE_INTEGER
```

Maximum:

```
Number.MAX_SAFE_INTEGER;  
    // 9007199254740991
```

Minimum:

```
Number.MIN_SAFE_INTEGER;  
    // -9007199254740991
```

---

## 37. `Number.isSafeInteger()`

```
Number.isSafeInteger(100);  
// true  
  
Number.isSafeInteger(Number.MAX_SAFE_INTEGER);  
// true
```

কিন্তু:

```
Number.isSafeInteger(Number.MAX_SAFE_INTEGER + 1);  
// false
```

খুব বড় integer-এর exact precision দরকার হলে `BigInt` ব্যবহার করা যেতে পারে।

## 38. Number Conversion Cheat Sheet

---

```
Number("100");  
// 100  
  
Number("10.5");  
// 10.5  
  
Number("100px");  
// NaN  
  
parseInt("10.99");  
// 10  
  
parseInt("100px");  
// 100  
  
parseFloat("10.99");  
// 10.99  
  
parseFloat("99.99px");  
// 99.99  
  
(99.456).toFixed(2);  
// "99.46"
```

---

## 39. কখন কোনটা ব্যবহার করবে?

---

### Number()

যখন পুরো value valid number কিনা convert করতে চাও:

```
const age = Number(input);
```

### parseInt()

যখন integer parse করতে চাও:

```
const count = parseInt(input, 10);
```

### parseFloat()

যখন decimal parse করতে চাও:

```
const price = parseFloat(input);
```

### toFixed()

যখন number-কে formatted decimal string হিসেবে দেখাতে চাও:

```
const formattedPrice = price.toFixed(2);
```

---

## 40. Common Mistake — parseInt() দিয়ে সব Conversion

---

অনেকে ভাবে:

```
parseInt("10.99");
```

মানে "10.99" কে number বানানো।

কিন্তু result:

```
10
```

যদি decimal preserve করতে হয়:

```
Number("10.99");  
// 10.99
```

অথবা:

```
parseFloat("10.99");  
// 10.99
```

---

## 41. Common Mistake — `toFixed()` এর Result Number ভাবা

---

```
const price = 99.99;

const formatted = price.toFixed(2);

console.log(typeof formatted);
// "string"
```

Calculation-এর জন্য দরকার হলে:

```
const number = Number(formatted);
```

কিন্তু display-এর জন্য string হওয়াই expected।

---

## 42. Common Mistake — User Input সরাসরি যোগ করা

---

```
const a = "10";
const b = "20";

console.log(a + b);
// "1020"
```

কারণ `+` string concatenate করতে পারে।

সঠিক:

```
console.log(Number(a) + Number(b));
// 30
```

---

## 43. Real-World Example — Shopping Cart

---

```
const price = "499.99";
const quantity = "2";

const total = Number(price) * Number(quantity);

console.log(total);
// 999.98
```

Display:

```
console.log(`Total: ₳${total.toFixed(2)}`);
```

Output:

```
Total: ₳999.98
```

---

## 44. Real-World Example — Form Validation

---

```
const age = Number("25");

if (!Number.isFinite(age)) {
  console.log("Invalid age");
} else if (!Number.isInteger(age)) {
  console.log("Age must be an integer");
} else {
  console.log("Valid age");
}
```

---

## 45. Real-World Example — Safe Numeric Conversion

---

একটা reusable helper:

```
function toNumber(value) {  
  const number = Number(value);  
  
  if (!Number.isFinite(number)) {  
    return null;  
  }  
  
  return number;  
}  
  
console.log(toNumber("100"));  
// 100  
  
console.log(toNumber("hello"));  
// null
```

## 46. `Number()` vs Unary `+`

দুটোই conversion করতে পারে:

```
Number("100");  
// 100  
  
+"100";  
// 100
```

কিন্তু readability-এর দিক থেকে:

```
Number(value)
```

অনেক সময় beginner এবং production code-এ বেশি explicit।

## 47. Conversion Decision Tree

User input:

```
const input = "123.45";
```

পুরো value valid number হতে হবে?

```
Number(input);
```

Integer দরকার?

```
parseInt(input, 10);
```

Decimal parse করতে হবে?

```
parseFloat(input);
```

শুধু display format করতে হবে?

```
number.toFixed(2);
```

# 48. Quick Comparison

| Method                 | Purpose                 | Return  |
|------------------------|-------------------------|---------|
| Number()               | Full numeric conversion | number  |
| parseInt()             | Integer parsing         | number  |
| parseFloat()           | Decimal parsing         | number  |
| toFixed()              | Decimal formatting      | string  |
| Number.isNaN()         | Actual NaN check        | boolean |
| Number.isFinite()      | Finite number check     | boolean |
| Number.isInteger()     | Integer check           | boolean |
| Number.isSafeInteger() | Safe integer check      | boolean |

## 49. Practice — Predict the Output

---

### Question 1

---

```
console.log(Number("50"));
```

### Question 2

---

```
console.log(Number("50px"));
```

### Question 3

---

```
console.log(parseInt("50.99", 10));
```

### Question 4

---

```
console.log(parseFloat("50.99px"));
```

### Question 5

---

```
console.log((10.567).toFixed(2));
```

### Question 6

---

```
console.log(typeof (10.567).toFixed(2));
```

### Question 7

---

```
console.log(Number.isNaN("hello"));
```

## Question 8

---

```
console.log(isNaN("hello"));
```

## Question 9

---

```
console.log(Number.isInteger(10.5));
```

## Question 10

---

```
console.log(Number.isFinite(Infinity));
```

---

# 50. Coding Practice

---

## Challenge 1 — Add Two Inputs

```
function add(a, b) {  
  // implement  
}  
  
console.log(add("10", "20"));  
// 30
```

---

## Challenge 2 — Parse Price

```
function getPrice(value) {  
    // return a valid number or null  
}  
  
console.log(getPrice("99.99"));  
// 99.99  
  
console.log(getPrice("hello"));  
// null
```

---

## Challenge 3 — Format Price

```
function formatPrice(price) {  
    // return something like "€99.99"  
}
```

---

## Challenge 4 — Validate Integer

```
function isValidAge(value) {  
    // value must be a finite integer  
}
```

Expected:

```
isValidAge("25");  
// true  
  
isValidAge("25.5");  
// false  
  
isValidAge("hello");  
// false
```

---

# 51. Interview Questions

---

## Beginner

1. JavaScript-এ integer এবং float কি আলাদা data type?
2. `Number()` কী করে?
3. `parseInt()` কী করে?
4. `parseFloat()` কী করে?
5. `toFixed()` কী return করে?
6. `Number("100px")` এর result কী?
7. `parseInt("100px")` এর result কী?
8. `0.1 + 0.2` কেন exactly `0.3` হয় না?

## Intermediate

9. `Number()` এবং `parseInt()` -এর পার্থক্য কী?
  10. `Number.isNaN()` এবং global `isNaN()` -এর পার্থক্য কী?
  11. `Number.isFinite()` কেন useful?
  12. `Number.isInteger()` কী check করে?
  13. `Number.MAX_SAFE_INTEGER` কী?
  14. Floating-point precision problem কী?
  15. `toFixed()` কেন calculation-এর মাঝখানে ব্যবহার করা উচিত নয়?
  16. User input থেকে আসা number কেন সাধারণত string হিসেবে handle করতে হয়?
- 

# 52. Mental Model

---

Numbers নিয়ে কাজ করার সময় এই flow মনে রাখো:

INPUT

↓

Usually String

↓

Conversion / Parsing

↓

Number

↓

Validation

↓

Calculation

↓

Formatting

↓

Display

Example:

```
const input = "99.99";

// 1. Convert
const price = Number(input);

// 2. Validate
if (!Number.isFinite(price)) {
  console.log("Invalid price");
}

// 3. Calculate
const total = price * 2;

// 4. Format
const displayPrice = total.toFixed(2);

// 5. Display
console.log(`Total: ₳${displayPrice}`);
```

এটাই real-world frontend development-এ খুব common pattern।

---

## 53. Golden Rules

---

1. `Number()` = full numeric conversion
2. `parseInt()` = integer parsing
3. `parseFloat()` = decimal parsing
4. `toFixed()` = formatting, এবং result string
5. User input/API data-এর type আগে check/convert করো
6. `Number.isNaN()` ব্যবহার করো actual `NaN` detect করতে
7. `Number.isFinite()` দিয়ে valid finite number check করা useful
8. `Number.isInteger()` দিয়ে integer validation করো
9. `0.1 + 0.2` ধরনের floating-point precision issue সম্পর্কে সচেতন থাকো
10. খুব বড় integer-এর exact precision দরকার হলে `BigInt` consider করো